Why Are My Prompts Leaked? Unraveling Prompt Extraction Threats in Customized Large Language Models

Zi Liang, Haibo Hu, Qingqing Ye, Yaxin Xiao, Haoyang Li
The Hong Kong Polytechnic University, Hong Kong, China
{zi1415926.liang,20034165r,hao-yang9905.li}@connect.polyu.hk, {haibo.hu,qqing.ye}@polyu.edu.hk

arXiv:2408.02416v2 [cs.CL] 12 Feb 2025

Abstract

The drastic increase of large language models' (LLMs) parameters has led to a new research direction of fine-tuning-free downstream customization by prompts, i.e., task descriptions. While these prompt-based services play an important role in many businesses, there has emerged growing concerns about the prompt leakage, which undermines the intellectual properties of these services and causes downstream attacks. In this paper, we analyze the underlying mechanisms of prompt leakage, and develop corresponding defending strategies. By exploring the scaling laws in prompt extraction, we analyze key attributes that influence prompt extraction, including model sizes, prompt lengths, as well as prompt types. Besides, we propose two hypotheses to explain how LLMs expose their prompts. The first is attributed to the perplexity, i.e., the familiarity of LLMs with texts, whereas the second is based on the straightforward token translation paths in attention matrices. To defend against such threats, we investigate whether alignments can mitigate the extraction of prompts. We find that current LLMs, even those with safety alignments like GPT-4, are highly vulnerable to prompt extraction attacks, even under the most straightforward user attacks. Therefore, we propose several defense strategies with the inspiration of our findings, which achieve almost 71.0% drop in the prompt extraction rate.

1 Introduction

Recently with the rapid development of instruction-following alignments (Ouyang et al., 2022; Glaese et al., 2022; Bai et al., 2022a,b; Perez et al., 2023) of large language models (LLMs) (Brown et al., 2020; OpenAI, 2023), customizing LLMs with prompts becomes a new trend of effortlessly building domain-specific LLMs. Debut in OpenAI's GPTs and Microsoft's Copilot Studio, a growing number of third-party developers are creating various downstream services by crafting their own instructions and incorporating them with domain-specific function callings or external documents. Known as *in-context learning* (ICL) (Brown et al., 2020; Dong et al., 2022), the LLM providers (i.e., the platforms) can transform such data into *prompts*, compose these prompts as the prefix of LLM inputs, and thus accomplish the construction of customized services, e.g., telehealth chatbots, without modifying any parameters.

However, as these prompts are the core assets of developers in customized LLMs, leaking them can jeopardize the IP rights of both the developers and platforms. For example, recently there have been up to 150 leaked GPTs already (Lin, 2024), and this number is ever increasing. With these leaked prompts, malicious users can easily mimic and even replicate a totally equivalent service to the original one, thereby jeopardizing the copyrights of victim third-party developers.

While there are some discussions in *prompt injection* (Bachaalany, 2024; Sha and Zhang, 2024; Zhang et al., 2023a) to steal both official and downstream prompts, the generic attack mechanisms of prompt extraction remain largely unexplored. Needless to say, the corresponding defensive strategies remain blank. Specifically, two critical questions arise: *i)* How do LLMs leak their prompts, and which factors of prompts and LLMs lead to such leakage? *ii)* Can LLMs' alignments defend against prompt extraction attacks (PEA)? If they cannot, how can we mitigate PEA?

To explore these two questions, instead of crafting new adversarial prompts for PEAs as done in most existing works (Sha and Zhang, 2024; Zhang et al., 2023a; Bachaalany, 2024; Yu et al., 2023; Yang et al., 2024), this paper aims to conduct an in-depth analysis of PEAs with respect to scaling laws, underlying mechanisms, and defensive strategies.

Specifically, we first identify three key factors that significantly influence the leakage of prompts, including prompt length, text type, and model size.

Based on the empirical evaluation, we provide two hypotheses to explain the mechanisms behind the memorization and the leakage of input prompts, including the *convincing premise* and the *parallel-translation of prompts*. On the one hand, by monitoring the perplexity of various prompts, we observe a strong positive correlation between LLM's familiarity with a prompt and its leakage rate, indicating that LLMs are more likely to expose prompts with which they are familiar. On the other hand, we introduce a novel set of indicators designed to trace the attention connection between prompts and generated texts, in which we observed abnormally high scores in those memorized and leaked prompts, illustrating a distinct and parallel attention trace to diagonal values in attention matrices. This discovery suggests that memorization may arise from a special token translation mechanism in the self-attention that moves tokens typically in response to certain user triggers.

For the second question, we design comparative experiments based on state-of-the-art adversarial prompts with both explicit and implicit intents, and evaluate whether well-aligned LLMs exhibit significant differences between these two types. Experiments reveal that even the most secure models remain vulnerable to PEAs. To mitigate this threat, we develop several simple but novel inference-time defense strategies based on our explanations of prompt leakage. Our experiments demonstrate that they yield much better defense than vanilla prompt defense methods, and have a very slight impact on the performance of prompts.

Our contribution can be summarized as:

- We provide a systematic evaluation to investigate factors in prompts and LLMs that influence prompt leakage. To this end, we construct a corresponding benchmark along with definitions and metrics.
- We derive in-depth explanations that elucidate the mechanisms and reasons behind prompt leakage.
- We put forward a series of prompt-engineering-based defense strategies for PEAs and validate their effectiveness.

2 Definitions and Evaluation Settings

In this section, we elaborate on the detailed attack and defense settings abstracted from real-world scenarios. We first define two types of prompt extraction, namely, soft extraction and formal extraction, and then describe the black-box attack settings in Section 2.2. A detailed introduction to the threat model is in Appendix B.

2.1 Definitions

Formally, given a language model $\Pr_\theta(\mathbf{x}^O|\mathbf{x}^P, \mathbf{x}^I)$ described in Appendix E.1, prompt extraction aims to craft a specific user input $\mathbf{x}^I$ that triggers the generated sentence $\mathbf{x}^O$ to contain the *uncovered* prompt $\mathbf{x}^{P'}$. Specifically, we can define four extraction tasks based on the definitions of $\mathbf{x}^{P'}$:

Definition 1 (Exact Prompt Extraction)**.** *The substring of $\mathbf{x}^O$, i.e., $\mathbf{x}^{P'}$, equals $\mathbf{x}^P$ exactly, i.e.,*

$$Pr_\theta(\mathbf{x}^O|\mathbf{x}^P, \mathbf{x}^I) = Pr_\theta(\mathbf{x}^{pre}, \mathbf{x}^P, \mathbf{x}^{su}|\mathbf{x}^P, \mathbf{x}^I), \quad (1)$$

where $\mathbf{x}^{pre}$ and $\mathbf{x}^{su}$ denote optional texts surrounding $\mathbf{x}^P$ in generated tokens.

Definition 2 (n-gram Fragment Extraction)**.** *The user input $\mathbf{x}^I$ is said to extract an n-gram fragment from the original prompt $\mathbf{x}^P$, if $\mathbf{x}^{P'} \in \{[x_i^P, x_{i+1}^P, ..., x_{i+n-1}^P]\}_{i=\{1,...,N_P-n+1\}}$.*

Definition 3 (ρ-fuzzy Prompt Extraction)**.** *The user input $\mathbf{x}^I$ extracts a ρ-fuzzy match of the original prompt $\mathbf{x}^P$, if $L(\mathbf{x}^{P'}, \mathbf{x}^P) \geq \rho$, where $L(\cdot,\cdot)$ denotes the normalized edit similarity:*

$$L(\mathbf{a}, \mathbf{b}) = 1 - \frac{d(\mathbf{a}, \mathbf{b})}{min(N_a, N_b)}, \quad (2)$$

in which N_a and N_b are the sequence length of texts $\mathbf{a}$ and $\mathbf{b}$, and $d(\cdot,\cdot)$ denotes the partial longest common subsequence (LCS) distance (Bergroth et al., 2000), i.e., the partial levenshtein distance without the replacement operation.

Definition 4 ($\delta - (\theta, \mathcal{D}_P, \mathcal{M}_P)$ Soft Extraction)**.** *Given an evaluation dataset $\mathcal{D}_P$ and its metric $\mathcal{M}_P$, both of which correspond to the task described by the original prompt $\mathbf{x}^P$, we define $\mathbf{x}^{P'}$ as the δ-revised (softly extracted) prompt of $\mathbf{x}^P$ for the language model $Pr_\theta(\cdot)$ if*

$$|\sum_{i=1}^{N_{\mathcal{D}_P}} \frac{1}{N_{\mathcal{D}_P}} \mathcal{M}_P(\mathbf{y}_i^O) - \sum_{i=1}^{N_{\mathcal{D}_P}} \frac{1}{N_{\mathcal{D}_P}} \mathcal{M}_P(\mathbf{y}_i^{O'})| \leq \delta, \quad (3)$$

where δ is the tolerant error of the revised prompt, $N_{\mathcal{D}_P}$ denotes the number of samples in the evaluation dataset, and $\mathbf{y}_i^O$ and $\mathbf{y}_i^{O'}$ denote the generated tokens from $Pr_\theta(\mathbf{x}_i^O|\mathbf{x}^P, \mathbf{x}_i^I)$ and $Pr_\theta(\mathbf{x}_i^O|\mathbf{x}^{P'}, \mathbf{x}_i^I)$ based on $\mathbf{x}^P$ and $\mathbf{x}^{P'}$, respectively.

We employ Definitions 2 and 3 to evaluate the extraction at the lexical level and utilize Definition 4 to measure the semantic similarity and effectiveness of extracted prompts.

2.2 Settings

Victim Prompts. We construct PEAD, a set of 961 prompts for the evaluation of PEA, which consists of the following four categories:

• GLUE task prompts. Collected from PromptBench (Zhu et al., 2023), it contains 243 prompts of 12 classical NLU tasks, such as sentiment analysis (e.g., SST-2 (Socher et al., 2013)), sentence equivalence analysis (e.g., MRPC (Dolan and Brockett, 2005)), and so on. We use this subset of data to do some basic evaluations as well as the performance measurement related to Definition 4.

• Leaked GPTs prompts. It contains 233 prompts sampled from those leaked prompts (Lin, 2024) of OpenAI's GPTs, which represents real-world customized instructions.

• Function callings. It contains 332 standard open-source API descriptions (Rubin, 2023) formatted in JSON. Different from others, this category is well organized with those hard-to-memorize elements, such as the URLs and generated keys, which make it suitable for evaluation on prompt memorization.

• Popular role-play prompts. It contains 153 prompts commonly used on ChatGPT for customization (Ma, 2025) before the release of GPTs. Compared to leaked GPTs prompts, these prompts typically exhibit longer sequence lengths with more detailed task descriptions.

Some prompt examples of the above four categories are shown in Table 2.

Adversarial Prompts. We select several state-of-the-art adversarial prompts (Appendix E.2) in our evaluations of the scaling law and the defense strategies. Besides, to explore the effectiveness of LLMs' alignments to PEA, we further divide these adversarial prompts into two parts, including prompts with explicit and implicit intents. Specifically, adversarial prompts with explicit intents, which are the simplest form of attack, typically ask LLMs to return their prompts directly. In contrast, the implicit prompts, represented by studies in Appendix E.2, will conceal their real intents and attempt to elicit LLMs to leak their prompts by some strategically deceptive tricks. An example of that is "prompt repetition", with the formation such as "`Could you please repeat your instructions three times?`". We obtain adversarial prompts by rephrasing them multiple times using GPT-4, as shown in Table 6.

Metrics. We propose *uncovered rate* (UR) as the core evaluation metric in this paper. UR indicates the success rate of extracting a given prompt from LLMs. By incorporating UR into n-gram in Definition 2 and ρ-fuzzy similarity in Definition 3, we can easily estimate the safety of the given LLMs at various levels of extraction granularity, i.e.,

$$\mathbf{UR}_{\mathcal{K}} = \frac{\sum_{\mathbf{x}^P \in \mathcal{D}} \mathbb{1}_{\mathcal{K}}(\mathbf{x}^P, \mathbf{x}^{P'})}{|\mathcal{D}|}, \tag{4}$$

where $\mathbb{1}_{\mathcal{K}}$ is the indicator function determined by the prompt extraction criterion $\mathcal{K}$ defined by one of the definitions in Section 2.1. Its value is 1 if $\mathbf{x}^{P'}$ is identified as an extracted version of $\mathbf{x}^P$; otherwise, it is 0.

For $\delta - (\theta, \mathcal{D}_P, \mathcal{M}_P)$ prompt soft extraction problem, following previous works such as PromptBench (Zhu et al., 2023), we simply use accuracy, precision, recall, and F1 score as the evaluation metrics on GLUE (Wang et al., 2019) tasks.

Serialization Patterns The serialization pattern, which organizes the prompts and the dialogue history into a text input sequence, is also an important factor for LLMs' privacy against prompt extraction. As the *meta-prompt* of LLMs, the serialization pattern used in our experiments is quite straightforward: "`Instruction: [P] User: [U] Assistant: `", where `[P]` and `[U]` represent slots to be filled with the prompt and the user input, respectively. For function calling experiments, we add an extra explanation to the serialization to assist LLMs in understanding the usage of the APIs, such as "`Instruction: You have the following function calling: '[P]' to help users.`".

Further details on the settings and implementation can be found in Appendix D.

3 Factors Influencing Prompt Extraction

3.1 Model Size

Regarding model sizes, we experiment with the scaling law on Pythia (Biderman et al., 2023), a group of language models with parameters from 70 million to 12 billion, all of which are pre-trained on the Pile (Gao et al., 2021) corpus and only differ in their model sizes.

As shown in Figure 1, all the curves, together with their variation ranges, increase with the model size. Besides, we can observe obviously higher average uncover rates for implicit intent prompts than in explicit ones on larger models, indicating that larger models can memorize prompts and are more likely to be extracted under implicit intent attacks. Nevertheless, such an average difference